

Group030_A1.zip

这是我们项目的数据，请你给我可以参考的 Regex 和 Unicode

思考了 37s

可以。根据你们 Group030_A1.zip 里的真实数据，我建议 Regex 和 Unicode 不要作为一个孤立的小检查，而是放进 **validation / cleaning evidence** 里面，重点证明你们处理了“格式不一致”和“多语言文本污染”。

你们这份数据里很适合做 Regex 的字段主要有 ID、日期、金额、百分比、postcode、语言代码、SKU、URL，以及 review/note 里的系统标签。Unicode 则重点放在 reviewText / Review_Text / reviewTitle / Review_Title，因为你们的数据确实包含英语、中文、日语、阿拉伯语、俄语、印地语等多语言文本，以及 emoji、©、™、特殊空格等字符。

1. ID 字段 Regex

你们的数据中 ID 格式非常规则，可以直接用：

字段	实际例子	推荐 Regex
customer_id	CUS00072	^CUS\d{5}\$
product_id	PRD0814	^PRD\d{4}\$
order_id	HORD004700	^HORD\d{6}\$
order_item_id	HITM0014798	^HITM\d{7}\$
delivery_id	HDEL004700	^HDEL\d{6}\$
review_id	HREV000801	^HREV\d{6}\$
supplier_id	SUP001	^SUP\d{3}\$

例如 Python：

```
import re
```

```
pattern = r"^CUS\d{5}$"
```

```
df["valid_customer_id"] = (  
    df["customer_id"]  
    .astype(str)  
    .str.match(pattern, na=False)  
)
```

HD 标准下，不要只写“用了 Regex”，建议同时输出 invalid rows：

```
invalid_customer_ids = df[  
    ~df["customer_id"].astype(str).str.match(r"^CUS\d{5}$", na=False)  
]
```

这样 report 里可以证明：

Customer identifiers were validated against the expected source-system identifier structure rather than merely checking for non-null values.

2. source_system_record_id

你们真实格式：

SRC-030-H-004700

SRC-030-H-002060

可以用：

`^SRC-030-H-\d{6}$`

Python：

```
r"^SRC-030-H-\d{6}$"
```

如果以后想写得稍微通用一点：

`^SRC-\d{3}-[A-Z]-\d{6}$`

但是针对你们 Group030，我更建议第一种，因为 validation 应该按照已知 source

contract 来做。

3. Product SKU

你们数据里有：

SKU-CAN00001

SKU-VEL00002

SKU-CAN00909

SKU-VEL01000

所以推荐：

```
^SKU-(?:CAN|VEL)\d{5}$
```

Python：

```
r"^SKU-(?:CAN|VEL)\d{5}$"
```

这个比：

```
^SKU-[A-Z]{3}\d{5}$
```

更严格，因为后者会把不存在的品牌代码例如：

SKU-ABC00123

也判为合法。

4. Coupon Code

你们数据里：

B2SAVE-82

B2SAVE-25

B2SAVE-65

而且允许为空。

因此：

```
^B2SAVE-\d{2}$
```

实际 validation 应该是：

```
valid_coupon = (  
    df["coupon_code"].isna()  
    | df["coupon_code"].eq("")  
    | df["coupon_code"].str.match(r"^B2SAVE-\d{2}$", na=False)  
)
```

注意这是一个很好的 HD 点：

nullable 和 **invalid format** 是两个不同概念。

不能用 Regex 把 null 直接判错，因为 dictionary 里面 coupon_code 是 nullable。

5. Australian postcode

你们实际值例如：

3122

3182

3053

3008

3070

最基本：

```
^\d{4}$
```

因为你们数据集中都是 Victoria，也可以进一步做 semantic validation：

```
postcode.str.match(r"^\d{4}$")
```

之后再检查范围，例如：

```
postcode_num = pd.to_numeric(df["home_postcode"], errors="coerce")
```

```
valid_vic_postcode = postcode_num.between(3000, 3999)
```

这就比单独 Regex 更好：

- Regex → syntactic validation
- range/domain → semantic validation

这句话很适合写进 report。

6. Language Code

实际值：

en

es

pl

de

ja

hi

ru

zh

ar

fr

it

pt

nl

Regex 可以：

`^[a-z]{2}$`

但是 HD 版本建议不要停在这里。

因为：

xx

qq

zz

也满足 Regex。

最好：

```
VALID_LANGUAGES = {
```

```
"en", "es", "pl", "de", "ja", "hi",  
"ru", "zh", "ar", "fr", "it", "pt", "nl"  
}
```

```
valid_language = df["language_code"].isin(VALID_LANGUAGES)
```

所以：

Regex validates structural conformity, while an allow-list validates domain membership.

7. 日期 Regex

你们两个 source 日期格式明显不一样。

JSON 中类似：

2014-09-05

2015-12-23

可以：

```
^\d{4}-\d{2}-\d{2}$
```

XML 里类似：

28/04/2018

01/05/2018

可以：

```
^\d{2}/\d{2}/\d{4}$
```

Timestamp XML：

25/04/2018 09:33:00

30/07/2018 18:26:00

可以：

```
^\d{2}/\d{2}/\d{4}\s\d{2}:\d{2}:\d{2}$
```

不过这里你们 report 最好强调：

Regex 只能证明字符串长得像日期，不能证明日期真的存在。

例如：

31/02/2018

Regex 一样通过。

因此真正 cleaning 应该：

```
pd.to_datetime(  
    series,  
    format="%d/%m/%Y",  
    errors="coerce"  
)
```

然后：

```
invalid_dates = series[parsed_dates.isna() & series.notna()]
```

这是明显比只做 Regex 更高质量的 validation。

8. 金额字段 Regex

你们 XML 中实际有：

AUD 622.90

AUD 3,323.97

AUD 11.39

AUD 1,311.55

因此非常适合：

```
^AUD\s(?:\d{1,3}(?:,\d{3})*|\d+)\.\d{2}$
```

例如：

```
money_pattern = r"^AUD\s(?:\d{1,3}(?:,\d{3})*|\d+)\.\d{2}$"
```

能接受：

AUD 622.90

AUD 3,323.97

AUD 1,311.55

不能接受:

AUD622.90

\$622.90

622.9

AUD 3,32.97

清洗:

```
clean_money = (  
    s.str.replace("AUD ", "", regex=False)  
    .str.replace(",", "", regex=False)  
)
```

```
numeric_money = pd.to_numeric(clean_money, errors="coerce")
```

9. Percentage

你们 Coupon_Discount:

0%

5%

10%

15%

20%

25%

Regex:

```
^\d{1,3}%$
```

更严格可以:

```
^(?:0|5|10|15|20|25)%$
```

如果 source contract 明确只有这些值, 我推荐第二种。

然后:


```
discount = (  
    s.str.rstrip("%")  
    .pipe(pd.to_numeric, errors="coerce")  
)
```

10. URLs

你们 review text 里面存在：

<https://reviews.example/verified?source=p3&lang=en>

<https://orders.example/help>

可以用于清理：

`https?://\S+`

Python：

```
text = text.str.replace(  
    r"https?://\S+",  
    "",  
    regex=True  
)
```

但是有一点非常重要：

不要把 URL Regex 用在原始数据覆盖式修改上。

建议：

review_body_raw

↓

review_body_clean

保留 raw source，清洗结果进入 canonical output。

11. HTML 标签

你们 review 里大量真实内容：

```
<blockquote data-lang="en">

<p>

<section class="customer-review">

<aside>

<div>

<br />
```

可以:

```
<[^>]+>
```

例如:

```
clean = s.str.replace(
    r"<[^>]+>",
    " ",
    regex=True
)
```

这在你们项目里非常值得做，因为 `review_body_clean` 本身就是 `data dictionary` 里的 canonical field。

12. System metadata

你们 `review` 里面还有:

```
[RATING: 5/5]
```

```
[SOURCE: mobile-app ©]
```

```
[VERIFIED_PURCHASE]
```

```
[SOURCE: web-form ™]
```

可以分别识别。

统一 bracket metadata:

```
\[[^\]]+\]
```

例如:

```
s.str.replace(r"\\[\\^\\]+\\", "", regex=True)
```

更严格：

```
\\(?:RATING:\\s*d/5|SOURCE:[\\^\\]+|VERIFIED_PURCHASE)\\
```

我更推荐后者，因为不会误删用户正文里正常的：

```
[important]
```

13. Reference 和 SKU metadata

你们 review 尾部真实出现：

Reference: HORD000573

SKU: SKU-VEL00076

以及：

Reference: HORD004188 SKU: SKU-VEL00388

可以提取：

```
Reference:\\s*(HORD\\d{6})
```

SKU：

```
SKU:\\s*(SKU-(?:CAN|VEL)\\d{5})
```

甚至可以拿它做 cross-source consistency validation：

```
reference_order = text.str.extract(  
    r"Reference:\\s*(HORD\\d{6})"  
)[0]
```

然后检查：

```
reference_order == review["order_id"]
```

这个是非常好的 HD validation，因为它不是简单格式检查，而是：

embedded metadata vs structured field consistency check。

Unicode 部分

这里你们的数据非常适合做，因为我实际检查到 review 里面存在大量非 ASCII 字

符。

例如：

中文

日本語

العربية

русский

हिन्दी

español

überwachung

cámara

以及：



™

©



所以最重要的一点是：

不要写成“删除所有非 ASCII 字符”

比如千万不要：

```
text.encode("ascii", "ignore").decode()
```

因为这样会把：

便利さと惜しい点が同居

直接毁掉。

也会破坏：

cámara

变成：

cmara

这是不正确的数据清洗。

14. Unicode normalization

我建议你们使用：

```
import unicodedata
```

```
def normalize_unicode(x):  
    if not isinstance(x, str):  
        return x  
    return unicodedata.normalize("NFKC", x)
```

然后：

```
df["review_body_clean"] = (  
    df["review_body_raw"]  
    .map(normalize_unicode)  
)
```

NFKC 可以帮助统一一些 compatibility characters。

比如 Unicode 中看起来一样但底层 code point 不一样的字符。

HD report 可以写：

Unicode NFKC normalisation was applied before text comparison to reduce visually equivalent but code-point-distinct representations, while preserving multilingual content.

15. Unicode whitespace

你们 review 里实际存在：

```
\t
```

`\n`

` `;

以及网页文本常见特殊空格。

建议：

```
import re
```

```
import html
```

```
def clean_whitespace(text):
```

```
    text = html.unescape(text)
```

```
    text = re.sub(r"\s+", " ", text)
```

```
    return text.strip()
```

其中：

` `;

经 `html.unescape()` 后通常会变成 non-breaking space:

`\u00A0`

然后再统一 whitespace。

也可以显式：

```
text = text.replace("\u00A0", " ")
```

16. Zero-width / invisible Unicode

这是 Unicode validation 中很适合写进报告的一项。

例如检查：

```
import unicodedata
```

```
def contains_invisible_unicode(text):
```

```
if not isinstance(text, str):  
    return False  
  
return any(  
    unicodedata.category(ch) == "Cf"  
    for ch in text  
)
```

Unicode Cf 是 Format characters, 例如潜在:

ZERO WIDTH SPACE

ZERO WIDTH JOINER

LEFT-TO-RIGHT MARK

RIGHT-TO-LEFT MARK

你们有 Arabic 等 RTL 文本, 所以这里不要无脑删除所有 format character。

正确做法应该是:

detect → inspect → selectively normalise

而不是:

detect → delete everything

17. Emoji detection

由于你们实际 review 里有:



如果这些是 source decoration, 而不是正文内容, 可以单独标记或清理。

简单 Regex:

```
emoji_pattern = (  
    r"[\U0001F300-\U0001FAFF"  
    r"\u2600-\u26FF"  
    r"\u2700-\u27BF]+"
```

然后：

```
text = re.sub(emoji_pattern, " ", text)
```

不过建议仅对：

```
review_body_clean
```

做，而原始 `review` 保留。

18. 多语言字符不要删

你们 `data dictionary` 有：

```
review_body_clean
```

```
review_body_latin_analysis
```

这个设计其实已经提示了应该区分：

`review_body_clean`

应该：

- 保留中文
- 保留日语
- 保留阿拉伯语
- 保留俄语
- 保留 Hindi
- 保留 accented Latin characters

例如：

`cámara`

`überwachung`

都应该保存。

review_body_latin_analysis

如果这是专门给 Latin-based 分析使用，可以单独产生。

例如：

```
def latin_only_analysis(text):  
    if not isinstance(text, str):  
        return ""  
  
    chars = []  
  
    for ch in text:  
        name = unicodedata.name(ch, "")  
  
        if (  
            "LATIN" in name  
            or ch.isspace()  
            or ch.isdigit()  
            or ch in ".,!?'-"  
        ):  
            chars.append(ch)  
  
    return "".join(chars)
```

注意：这只能用于 derived analysis field。

不能覆盖原始 multilingual review。

19. Control characters

建议查：

```
def find_control_chars(text):
    if not isinstance(text, str):
        return []

    return [
        (repr(ch), unicodedata.name(ch, "UNKNOWN"))
        for ch in text
        if unicodedata.category(ch).startswith("C")
        and ch not in "\n\r\t"
    ]
```

因为有些字符串会存在不可见控制字符，视觉上看不出来，但会导致：

```
"value" != "value\u200b"
```

从而破坏：

- duplicate detection
- join
- PK/FK
- equality comparison

20. 推荐你们直接做一个 **Regex validation table**

如果你们现在正在做 validation，我建议至少放这些：

Validation	Field	Regex / Rule	Purpose
Customer ID	customer_id	^CUS\d{5}\$	Identifier structure
Order ID	order_id	^HORD\d{6}\$	Identifier structure

Validation	Field	Regex / Rule	Purpose
Item ID	order_item_id	^HITM\d{7}\$	Identifier structure
Product ID	product_id	^PRD\d{4}\$	Identifier structure
Delivery ID	delivery_id	^HDEL\d{6}\$	Identifier structure
Review ID	review_id	^HREV\d{6}\$	Identifier structure
Supplier ID	supplier_id	^SUP\d{3}\$	Identifier structure
SKU	product_sku	^SKU-(?:CAN VEL)\d{5}\$	SKU contract
Source ID	source_system_record_id	^SRC-030-H-\d{6}\$	Source-system contract
Postcode	home_postcode	^\d{4}\$	Basic postcode syntax
Language	language_code	^[a-z]{2}\$ + allow-list	ISO-like code validation
XML date	date fields	^\d{2}/\d{2}/\d{4}\$	Source date structure

Validation	Field	Regex / Rule	Purpose
JSON date	date fields	<code>^\d{4}-\d{2}-\d{2}\$</code>	Source date structure
XML money	monetary fields	<code>^AUD\s(?:\d{1,3}(?:\d{3})* \d+)\.\d{2}\$</code>	Currency parsing
Discount	Coupon_Discount	<code>^(?:0 5 10 15 20 25)%\$</code>	Percentage format

Unicode 我建议你们至少写这 5 个检查

1. **NFKC Unicode normalization**
2. **HTML entity decoding**, 例如 ` `;
3. **Whitespace normalization**, 包括 `\t` / `\n` / `NBSP`
4. **检测 invisible/control Unicode**
5. **保留 multilingual text**, 禁止简单 **ASCII stripping**

你们这份数据特别适合把 Regex 和 Unicode 放到一起写成一条完整 cleaning pipeline:

raw

- Unicode NFKC
- HTML unescape
- remove source metadata
- remove HTML tags
- remove URLs
- normalise whitespace
- preserve multilingual characters
- validate extracted IDs / references

→ output *_clean

这比单纯写一句“我们使用 Regex 清理数据”更符合 HD 的思路，因为它说明了**为什么清理、清理什么、保留什么，以及怎么验证清理没有破坏有效数据。**